Cool things to surf on MATLAB and GNU Octave.

Date: 16-Jul-2025

Author: Daniel Perales Ponce

1. Interpolation mechanism on MATLAB/GNU Octave.

I am discussing with y'all this wonderful mathematical tool in the context of this very powerful math machines we use for science and data processing almost everytime as it's really cool to do all this! For now, we are pretty much not doing that much of a difficult set of stunts to come up with something flashy, we are rather just providing this with some fairly simple usecases.

1.1. Random soft curves generator.

Let's begin with a 5 by 5 grid with this random pack of data this way. First, we call `t` an interval from 1 to 5, which is going to set the size of our small grid. Then, we use `meshgrid` to create said plane. Our data sample is a random 5 by 5 noise, and we are going to interpolate it with MATLAB's builtin function into a 40 by 40 meshgrid; thus transforming: **small** mat -> **Big** mat.

Now, to display the data we made, we might want to `imagesc` our `Z` matrix, so to watch what we are going from and what are we now composing based off it.

After this, we can compare in a figure each thing, to see how did the process transform the original noise sample into some meaningful data, could it be useful for us at any given point

```matlab
t = 1:5;
T = linspace(1,5,40);
[X, Y] = meshgrid(t,t);
Z = rand(5,5);
[X1, Y1] = meshgrid(T,T);
Z1 = interp2( X,Y,Z, X1,Y1);
% is up to you choosing the method:

subplot(2,2,1)
imagesc(Z); colormap('hot'); colorbar;
axis('equal')
subplot(2,2,2)
imagesc(Z1); colormap('hot'); colorbar;
axis('equal')
subplot(2,2,3:4)
surf(X1,Y1,Z1); colormap('hot'); hold on
contour(X1,Y1,Z1)
```

Now, I'm showing Figure 1, which is an output figure produced by the software GNU Octave when prompted these commands in the terminal. I used a **cubic interpolation** in this one. Depending on the interpolation method, you might get to see an smoothed curve or rather an angular one, whether is a cubic or linear method respectively.

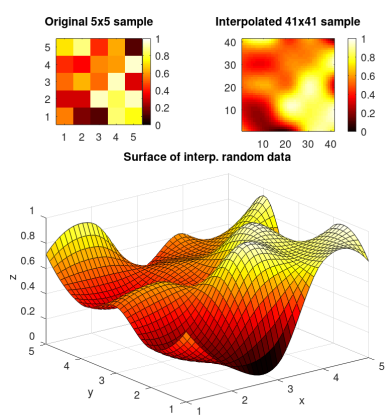

Figure 1: Graph rendered with cubic interpolation.

Whereas in the Figure 2 I show a **linear** interpolation, which usually fills the space with planes that describe the original data sample. Seeing the scanned image of matrix **Z** we can certainly refer how the surface is made out of intersecting planes, instead.

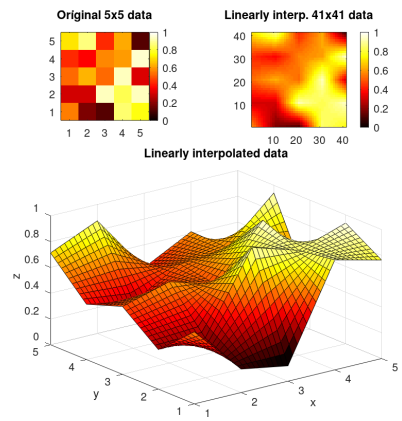

Figure 2: Graph rendered using linear interpolation.

2. Directed flat waves as cosines.

While still working on something new, I sumbled across some curious realization while mathematically trying to adress the parameters of a cosine function, in which I accidentally inputed this details and suddenly it turned into this formula:

$$\vec{Z}(\hat{x}, \hat{y}) = z \cos[f(A\hat{x} + B\hat{y})]\hat{z}$$

In which, believe it or not, we can say the $\hat{x}$ and $\hat{y}$ vectors are actually the meshgrid objects, f is the frequency and A, B are the components of the direction vector of the plane that sits perpendicular to the wave front.

Based on this, I decided to put this into a full standard MATLAB function, by writing all of these details into it.

For keeping it simple, I kept frequency as *natural* and assumed direction angle would be provided in degrees; while I ask for the x and y meshgrid objects to porperly locate the values of my function in the space.

```matlab
function Z = wave_dir(X, Y, freq, dir)

% Be X and Y two meshgrid matrices,
% and Z its correspondent value.
%
% dir is an angle to which the wave
% is directed.
%
% freq is the wave s frequency

h = cosd(dir); k = sind(dir);

Z = cos( freq.*(h.*X + k.*Y) );
```

As a way of testing the outcomes, I decided to plot a surface graph, so to see how would certain waves colliding at different angles make a interference pattern, because of the sum of their peaks and lows in certain points.

In Figure 3, we see an example of this pattern, both in the form of a `imagesc` and at the bottom image we see the surface graphic of the resultant wave of the sum of two colliding waves. They have different frequencies, and they are pointed to 20deg and 60deg respectively.

To further test my function, I tried to collide two perpendicular waves, pointing 0 and 90 degrees each one. We see this in Figure 4, in which the first wave has a frequency of 1, and the second a frequency of 3.

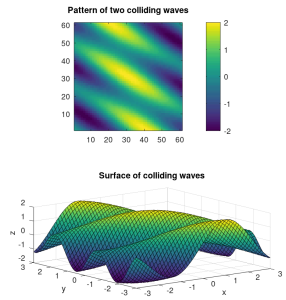

Figure 3: Example of the sum of 2 waves.

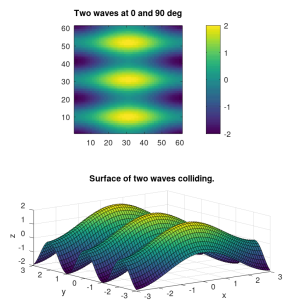

Figure 4: Example of perpendicular waves.

3. The play of the discussion of my mind logs.

3.1. 16-Jul-2025

I said---Try to find that **interp** function inside of Octave, I know I used it before!---hour: 15:30 approximately---it was **interp2**!

---Found it!---at 15:45 approx.

I entried---Finally, we have an functioning applet to porperly format codeblocks, which were certainly a headache to deal with---at 23:53 of 16-Jul-2025.

3.2. 17-Jul-2025

I entried---Maybe it's time to write my newest finding on Octave, which was writing the function to define colliding directional waves! Let's see how it goes---at 13:55 today.

---It did it!---I screamed as loudly as I could, being it 23:55 of today---I wrote down the waves section!